MP5: Camera View and Manipulation

Due Time: Refer to the course web-site

Objective
In this programming assignment, we want to further verify our understanding of pivoted transformation, and basic camera manipulation.

Approach
We will integrate camera view and manipulation into our solutions from MP5 (you are welcome to use my MP5 solution). Here is my attempt at implementation for your reference.

Scene Hierarchy: Your MP5 hierarchy must be functioning completely (you can use my solution), where your user must be able to select and rotate any of the SceneNode in your hierarchy. This is an important prerequisite to verify that your camera view control is functioning properly.

Camera manipulation: To verify our understanding of how to manipulate the main camera,

- You will display a ***look at*** position in the scene.
  - You user will have slider-bar control to change the x/y/z values of this position.
- **Tumble**: *This is covered in our lectures*
  - Alt-Left mouse X/Y movement controls the horizontal and vertical tumble.
- **Track**: You must support tracking of the camera along the up and right direction of the camera.
  - Alt-Right mouse X/Y movement controls the right/up tracking
- **Dolly**: You must support zooming into/away of the camera
  - Alt-Scroll-Wheel zooms the camera towards/away from the look at position.

Small view camera: You will *compute* and *attach* a camera at the *front* of a leave node in the hierarchy (e.g., the Head scene node in my case). This camera:

- **Camera position**: always follow the front of the Head scene node in the hierarchy (must consider all of the parents' transforms).
- **Viewing direction** must be along the Head's *front* direction
  - You must display the viewing direction with a visible LineSegment
- **Up vector** must be aligned with the y-axis of the SceneNode
- **FOV** of 45 degrees
- **Viewport**: 30% width and height of the main display.
- **Decoration objects:** You must create at least 12 decoration objects to surround your scene hierarchy. These objects must be identifiably different (e.g., do not create 12 identical spheres). We will use these object to verify the correctness of the small view camera parameter computations.
  - **Note:** The primitives in the scene hierarchy are transformed drastically different from the default Unity transform. The default camera *frustum culling* assumes the default Unity transforms. This means, as camera moves around in the scene, scene node primitives may disappear (culled) seemingly randomly. For this reason, we will use these decoration objects for verification of the small camera views.

Finally, you must support a **reset** button to reset your scene to the initial position (I did not support this. Sorry).

Hints:

1. Remember to BACKUP often!! I back up by duplicating the entire project folder, about every 20-30 minutes!!
   a. I had to go back to previous saved versions, VERY often.

Credit Distribution:

1.	Does not include a proper scene for 3D viewing: a. A functioning hierarchy, with min of 3-levels, i. E.g., one similar to that from MP4 is good b. In addition to the hierarchy, include decoration objects that are sufficient to distinguish 3D views i. E.g., the color cylinders from MP4 is good		-80%
2.	Camera Manipulations a. X/Y/Z sliders LookAt position control b. Tumble: covered in lectures c. Track in horizontal/vertical directions d. Dolly to zoom in/out	 15% 0% 10% 5%	30%
3.	Small View Camera a. Position follows Head of the hierarchy b. Orientation follows Head rotation of the hierarchy c. Proper viewport for the camera d. Proper decoration objects (12 differentiable objects) e. Viewing direction LineSegment **Unable to select SceneNode or rotate the SceneNode hosing the small view camera: -35%**	 10% 10% 5% 10% 10%	40%
4.	**Small View Camera re-attachment (CSS551 Only)** **a. Able to attach on any currently selected scene node** **b. Reasonable viewing position + direction** **c. Follow the orientation and position of the attached**	 **10%** **10%** **10%**	**20%**
5.	Reset Button a. Reset to original position	 5%	5%
6.	Submission: zipfile content/name, Windowed, Resizable, EXE folder		5%

Bold Green fonts indicates CSS551-specfic requirements, CSS451 students: extra credit, you *do not* need to work on these.

Note:

- **CSS551 students:** your score will be based 100
- **CSS451 students:** your score will be based 80 converted to percentage. If you attempt small view camera attachment, the 20-point credits will go towards your extra credits for the class

For this assignment, you CANNOT use: *these are closely related to the functionality we want to practice*

Transform.RotateAround()
Transform.LookAt()

Quaternion.LookRotation()
Quaternion.SetLookRotation()

Be aware of the Unity Frustum culling, since we high-jacked the vertex shader transform, as far as Unity is concerned, all primitives are located at the origin! If you move the look at position up-wards, the hierarchy will start to disappear! ☺.

Warning: You will receive an automatic zero for the entire assignment if you use any of Unity's shader in your scene hierarchy. Make sure to talk to me if you see the need to use a shader that is very different from what we have developed in class. Once again, there is a relatively small amount of code to be written, BUT, they are tricky. Start early! Let me know your questions so that I can start looking into your problems as soon as you encounter any!

Extra Credit: Some ideas, DO NOT work on the following until you are completely done with the base assignment.

- How about multiple small view cameras, e.g., allow user to create/delete
- Multiple main camera under user control
- This is challenging and interesting:
    - Define a camera on the Head that tracks a position (view of this camera follows a user control position)
- 3rd person view behind the hierarchy (when the hierarchy is transformed, the 3rd person camera will follow always maintaining a fixed relative relationship, I think much like 3rd person view in a 1st person shooter type games). How can you make this 3rd person camera movement "fluid"?